# SPM UNIT-3

---

1. Explain how schedule risks are evaluated and mitigated. Include examples of common risks like resource unavailability, scope creep, or underestimated timelines.

A. **1. Evaluating Schedule Risks**

Project managers use structured methods to identify and measure risks that could delay timelines:

- **Risk Identification** Brainstorming sessions, checklists, and expert judgment are used to list possible risks (e.g., staff absence, requirement changes, unrealistic estimates).
- **Historical Data and Lessons Learned** Comparing current estimates with past projects to detect patterns of underestimation.
- **PERT (Program Evaluation and Review Technique)** Uses three estimates for each task: optimistic, pessimistic, and most likely. This helps quantify uncertainty in timelines.
- **Critical Path Method (CPM)** Identifies tasks that directly determine the project's end date. Any delay in these tasks will delay the whole project.
- **Probability-Impact Matrix** Risks are rated by likelihood and severity. High-probability, high-impact risks are prioritized for mitigation.

**2. Mitigating Schedule Risks**

Mitigation strategies aim to reduce either the likelihood or the impact of delays:

- **Resource Unavailability**
  - Cross-train team members so others can step in.
  - Maintain backup staff or contractors.
  - Use flexible scheduling to redistribute workload.
- **Scope Creep**
  - Enforce a strict change control process.
  - Require stakeholder approval for new features.
  - Prioritize requirements to ensure critical features are delivered first.
- **Underestimated Timelines**
  - Add contingency buffers to schedules.
  - Use iterative planning (Agile sprints) to re-estimate regularly.
  - Apply historical data to adjust overly optimistic estimates.

**3. Continuous Monitoring**

Even after mitigation plans are set, managers must track progress:

- **Burn-down charts** in Agile projects to visualize remaining work.
- **Earned Value Analysis (EVA)** to measure schedule variance.
- **Regular status reviews** to detect early warning signs of slippage.

2. What are the key components of a risk management plan in software project management?

A. **1. Risk Identification**

- Define the process for discovering potential risks.

- Use techniques like brainstorming, checklists, expert interviews, and historical data.
- Document risks related to scope, technology, resources, vendors, and compliance.

**2. Risk Analysis**

- **Qualitative Analysis**: Assess probability and impact using scales (e.g., high/medium/low).
- **Quantitative Analysis**: Use numerical models (e.g., Expected Monetary Value, Monte Carlo simulation).
- Prioritize risks based on exposure or severity.

**3. Risk Register**

- Central document listing:
  - Risk description
  - Category (technical, external, organizational, etc.)
  - Probability and impact
  - Owner and response strategy
  - Status and tracking notes

**4. Risk Response Planning**

- Define strategies for each risk:
  - **Avoid**: Change scope or approach to eliminate the risk.
  - **Transfer**: Shift risk to a third party (e.g., insurance, outsourcing).
  - **Mitigate**: Reduce probability or impact through preventive actions.
  - **Accept**: Acknowledge the risk and prepare contingency plans.

**5. Contingency and Fallback Planning**

- Allocate time, budget, or resources to handle risks if they materialize.
- Define fallback actions for high-impact risks.

**6. Risk Monitoring and Control**

- Establish procedures for tracking risks throughout the project.
- Use risk metrics, status reports, and review meetings.
- Update the risk register and response plans as needed.

**7. Roles and Responsibilities**

- Assign risk ownership to specific team members.
- Define escalation paths and decision-making authority.

**8. Communication Plan**

- Specify how risk information will be shared with stakeholders.
- Include frequency, format, and channels for reporting.

3. Explain probability-impact matrices, risk exposure, and risk prioritization. Include examples of how risks are ranked and handled based on severity.

A. **1. Probability-Impact Matrix**

This is a tool used to evaluate risks by plotting them on two dimensions:

- **Probability (Likelihood)**: How likely the risk is to occur (low, medium, high).
- **Impact (Severity)**: How serious the consequences would be if it occurs (low, medium, high).

The matrix helps visualize which risks are critical.

- **High probability + high impact** → top priority risks.
- **Low probability + low impact** → often accepted or monitored.

**Example:**

- Resource unavailability: High probability, high impact → critical risk.
- Minor documentation delay: Low probability, low impact → low-priority risk.

**2. Risk Exposure**

Risk exposure quantifies the "weight" of a risk. It is often calculated as:

$$\text{Risk Exposure} = \text{Probability} \times \text{Impact}$$

This gives a numerical value to compare risks.

- If probability = 0.7 (70%) and impact = 50 days delay, exposure = 35 days expected delay.
- If probability = 0.2 (20%) and impact = 100 days delay, exposure = 20 days expected delay.

So even though the second risk has a larger impact, the first one is more urgent because its expected effect is greater.

**3. Risk Prioritization**

Once risks are evaluated, they are ranked to decide which ones need immediate attention.

- **High exposure risks** → mitigation plans are mandatory.
- **Medium exposure risks** → monitored and controlled.
- **Low exposure risks** → often accepted as part of project uncertainty.

**Examples of prioritization and handling:**

- **Scope creep**: Medium probability, high impact → prioritized for strict change control.
- **Resource unavailability**: High probability, high impact → prioritized for backup staffing and cross-training.
- **Underestimated timelines**: Medium probability, medium impact → handled with contingency buffers and iterative re-estimation.
- **Minor UI bug delays**: Low probability, low impact → accepted and monitored.

4. Describe various techniques used for identifying risks in software project management. Include brainstorming, checklists, expert judgment, SWOT analysis, and historical data.

A. **1. Brainstorming**

- **How it works**: The project team gathers to openly discuss possible risks. Ideas are captured without judgment, then categorized later.
- **Strengths**: Encourages creativity, surfaces risks that might not appear in formal analysis.
- **Example**: During planning, developers suggest risks like "integration issues with legacy systems" or "delays due to unclear client feedback."

**2. Checklists**

- **How it works**: Managers use predefined lists of common risks based on industry standards or past projects.

- **Strengths**: Ensures no typical risks are overlooked; efficient for routine projects.
- **Example**: A checklist might include categories like staffing, technology, requirements, testing, and vendor dependencies.

**3. Expert Judgment**

- **How it works**: Experienced project managers, technical leads, or consultants provide insights based on prior projects.
- **Strengths**: Leverages domain expertise; useful when the team lacks experience.
- **Example**: An expert warns that "third-party API integration often causes unexpected delays" based on past experience.

**4. SWOT Analysis**

- **How it works**: Risks are identified by analyzing project **Strengths, Weaknesses, Opportunities, and Threats**.
- **Strengths**: Provides a structured view, linking internal and external factors.
- **Example**:
  - Strength: Skilled team.
  - Weakness: Limited testing resources.
  - Opportunity: New market adoption.
  - Threat: Competitor releasing similar software earlier.

**5. Historical Data**

- **How it works**: Past project records, lessons learned, and metrics are reviewed to identify recurring risks.
- **Strengths**: Evidence-based; helps avoid repeating mistakes.
- **Example**: If previous projects consistently underestimated testing time, managers flag "testing overruns" as a likely risk.

5. Explain the process of risk management in software projects. Provide examples from real or hypothetical projects.

A. **1. Risk Identification**

The first step is to recognize possible risks. Techniques include:

- **Brainstorming** with the team to surface issues like unclear requirements or integration challenges.
- **Checklists** of common risks (scope creep, resource shortages, vendor delays).
- **Expert judgment** from experienced managers or consultants.
- **SWOT analysis** to capture internal weaknesses and external threats.
- **Historical data** from past projects to highlight recurring problems.

**Example:** In a mobile banking app project, risks identified included regulatory compliance changes, third-party API instability, and potential delays in user interface design.

**2. Risk Analysis**

Once identified, risks are evaluated for **probability** (likelihood) and **impact** (severity).

- **Qualitative analysis**: Using a probability-impact matrix to categorize risks as

low, medium, or high.
- **Quantitative analysis**: Calculating risk exposure (probability × impact) to assign numerical values.

**Example:**
- Scope creep: Medium probability, high impact → significant risk.
- Developer illness: High probability, medium impact → moderate risk.
- Minor documentation delay: Low probability, low impact → negligible risk.

**3. Risk Prioritization**

Risks are ranked so that resources focus on the most critical ones.
- **High exposure risks** require immediate mitigation plans.
- **Medium risks** are monitored with contingency options.
- **Low risks** may be accepted as part of project uncertainty.

**Example:** In an e-commerce platform project, integration with a payment gateway was prioritized because failure would block core functionality, while minor UI delays were deprioritized.

**4. Risk Mitigation and Response**

Strategies include:
- **Avoidance**: Changing plans to eliminate the risk (e.g., using a proven technology instead of an experimental one).
- **Mitigation**: Reducing probability or impact (e.g., cross-training staff to handle resource unavailability).
- **Transfer**: Shifting responsibility (e.g., outsourcing testing).
- **Acceptance**: Acknowledging the risk and preparing contingency plans.

**Example:**
- Resource unavailability → backup developers and flexible scheduling.
- Scope creep → strict change control and stakeholder sign-off.
- Underestimated timelines → adding buffer time and iterative re-estimation.

**5. Monitoring and Control**

Risk management is continuous. Managers track risks throughout the project lifecycle using:
- **Burn-down charts** in Agile projects.
- **Earned Value Analysis (EVA)** for schedule and cost variance.
- **Regular risk reviews** to update the risk register.

**Example:** In a healthcare software project, weekly risk reviews helped detect early signs of testing delays, allowing the team to add extra testers before deadlines slipped.

6. Describe how the integration of WBS, scheduling, and network techniques supports successful project planning and execution.

A. **1. Work Breakdown Structure (WBS)**
- **Definition**: WBS decomposes the project into smaller, manageable components (deliverables, tasks, sub-tasks).
- **Purpose**: Provides clarity on scope, responsibilities, and deliverables.
- **Example**: In a software project, WBS might break down into modules like *UI*

*design*, *database setup*, *API integration*, *testing*, and *deployment*.

**2. Scheduling**

- **Definition**: Scheduling assigns timelines and resources to tasks identified in the WBS.
- **Purpose**: Ensures tasks are sequenced logically and deadlines are realistic.
- **Techniques**: Gantt charts, milestone charts, and time estimation methods (PERT, expert judgment).
- **Example**: UI design scheduled for weeks 1–3, database setup for weeks 2–4, testing for weeks 5–6.

**3. Network Techniques**

- **Definition**: Network techniques (like CPM and PERT) map task dependencies and identify the critical path.
- **Purpose**: Helps visualize task relationships, detect bottlenecks, and calculate project duration.
- **Example**: Testing cannot begin until both UI design and database setup are complete. The critical path shows the minimum project duration if all critical tasks stay on track.

**4. Integration of WBS, Scheduling, and Network Techniques**

When combined, these tools create a **cohesive project plan**:

- WBS defines **what** needs to be done.
- Scheduling defines **when** tasks will be done and by **whom**.
- Network techniques define **how tasks depend on each other** and identify critical sequences.

This integration ensures:

- **Clarity of scope**: No deliverable is overlooked.
- **Realistic timelines**: Tasks are sequenced and resourced properly.
- **Risk management**: Critical path analysis highlights tasks where delays would affect the entire project.
- **Efficient monitoring**: Progress can be tracked against WBS deliverables, scheduled milestones, and network dependencies.

**5. Example Case**

Suppose a company is developing an e-commerce platform:

- **WBS**: Breaks project into modules (product catalog, shopping cart, payment gateway, testing).
- **Scheduling**: Assigns durations (catalog: 2 weeks, cart: 3 weeks, payment: 4 weeks, testing: 2 weeks).
- **Network techniques**: Show that payment gateway integration is on the critical path; any delay here pushes the entire project.

By integrating all three:

- Managers know exactly what tasks exist, when they must be completed, and which ones are most critical.
- Resources can be allocated efficiently, and risks (like payment gateway delays) can be mitigated early.

7. Describe how critical activities influence the overall schedule and how project managers monitor them. Provide an example using a network diagram.

A. **1. Critical Activities and Their Influence**

- **Definition**: Critical activities are tasks that lie on the *critical path* of a project network.
- **Influence**:
  - Any delay in a critical activity directly delays the entire project.
  - Non-critical activities may have *float/slack* (extra time), but critical ones have zero slack.
  - Managers must allocate resources and attention to ensure these tasks stay on track.

**Example**: In a software project, "Database setup" and "Payment gateway integration" may be critical activities. If either slips, the final delivery date slips.

**2. How Project Managers Monitor Critical Activities**

- **Critical Path Method (CPM)**: Identifies the sequence of tasks with zero slack.
- **Regular Tracking**: Using Gantt charts, milestone reviews, or Earned Value Analysis to check progress.
- **Risk Management**: Adding buffers or contingency plans for critical tasks.
- **Resource Allocation**: Ensuring skilled staff and sufficient time are dedicated to critical activities.

**3. Example with a Network Diagram**

Imagine a simplified project to build a mobile app:

**Activities and Durations**

- A: Requirements (2 weeks)
- B: UI Design (3 weeks)
- C: Database Setup (4 weeks)
- D: API Integration (2 weeks)
- E: Testing (3 weeks)
- F: Deployment (1 week)

**Dependencies**

- A → B and C
- B → D
- C → D
- D → E → F

**Critical Path Calculation**

- Path 1: A → B → D → E → F = 2 + 3 + 2 + 3 + 1 = 11 weeks
- Path 2: A → C → D → E → F = 2 + 4 + 2 + 3 + 1 = 12 weeks

So the **critical path is A → C → D → E → F (12 weeks)**.

- Activities C, D, E, F are critical.
- If "Database Setup" (C) is delayed by 1 week, the entire project extends to 13 weeks.

**4. Monitoring in Practice**

- Managers track progress of C, D, E, F daily or weekly.
- They may assign extra staff to "Database Setup" to reduce risk.

- Non-critical tasks like UI Design (B) can slip slightly without affecting the final deadline, but critical tasks cannot.

8. Define float in project scheduling. What is the significance of activity float and how is it calculated?

A. **1. Definition of Float**

In project scheduling (PERT/CPM networks), **float** (also called *slack*) is the amount of time an activity can be delayed **without delaying the project's completion date**.

- If an activity has **zero float**, it lies on the **critical path**.
- If it has **positive float**, you have flexibility in scheduling it.

**2. Significance of Activity Float**

Float is important because it:

- Helps identify **critical vs. non-critical activities**.
- Provides **flexibility** in resource allocation (you can shift resources from high-float tasks to critical tasks).
- Assists in **risk management** — delays in high-float activities are less dangerous.
- Guides **project control** — managers monitor activities with low float more closely.

**3. Calculation of Float**

There are two main types:

- **Total Float (TF):**

$TF$ = Late Start (LS) − Early Start (ES)or$TF$ = Late Finish (LF) − Early Finish (EF)

→ It tells how much an activity can be delayed without affecting the project completion.

- **Free Float (FF):**

$FF$ = Earliest Start of successor − Early Finish of activity

→ It tells how much an activity can be delayed without affecting the start of its immediate successor.

9. Explain how to identify the critical path, calculate activity float, and determine critical activities. Use an example network diagram to illustrate the process.

A. **1. Steps to Identify the Critical Path**

1. **List activities** with their durations and dependencies.
2. **Draw the network diagram** (nodes or arrows showing precedence).
3. **Forward pass** → calculate **Earliest Start (ES)** and **Earliest Finish (EF)**.
   - $EF = ES +$ Duration
4. **Backward pass** → calculate **Latest Start (LS)** and **Latest Finish (LF)**.
   - $LS = LF -$ Duration
5. **Calculate float** for each activity:
   - Total Float $= LS - ES = LF - EF$
6. **Critical path** = sequence of activities with **zero float**.

**2. Example Network Diagram**

| Activity | Duration (days) | Predecessor |
|---|---|---|
| A | 3 | — |
| B | 4 | A |
| C | 2 | A |
| D | 5 | B, C |
| E | 3 | D |

**Network flow:**

- Start → A → B → D → E → Finish
- Start → A → C → D → E → Finish

**3. Forward Pass (ES & EF)**

- A: ES=0, EF=3
- B: ES=3, EF=7
- C: ES=3, EF=5
- D: ES=max(7,5)=7, EF=12
- E: ES=12, EF=15

So **project completion time = 15 days**.

**4. Backward Pass (LS & LF)**

- E: LF=15, LS=12
- D: LF=12, LS=7
- B: LF=7, LS=3
- C: LF=7, LS=5
- A: LF=min(3,5)=3, LS=0

**5. Float Calculation**

- A: TF = LS–ES = 0 → **Critical**
- B: TF = 0 → **Critical**
- C: TF = 2 (LS=5 – ES=3) → Non-critical
- D: TF = 0 → **Critical**
- E: TF = 0 → **Critical**

**6. Critical Path**

The critical path is: **A → B → D → E** (total duration = 15 days).
Activity C has **float = 2 days**, meaning it can be delayed up to 2 days without affecting project completion.

10. Explain how Gantt charts are used for planning, tracking progress, and managing time. Include an example of a Gantt chart representing a software development life cycle.

A. Gantt charts are valuable tools in software project management, offering a visual representation of project schedules, facilitating planning, progress tracking, and time management. They clearly display tasks, their durations, and dependencies, enabling teams to visualize the project's timeline and identify potential roadblocks.

**Use in Software Project Management:**

• **Planning:** Gantt charts help break down the software development lifecycle into manageable tasks, estimate task durations, and establish dependencies between

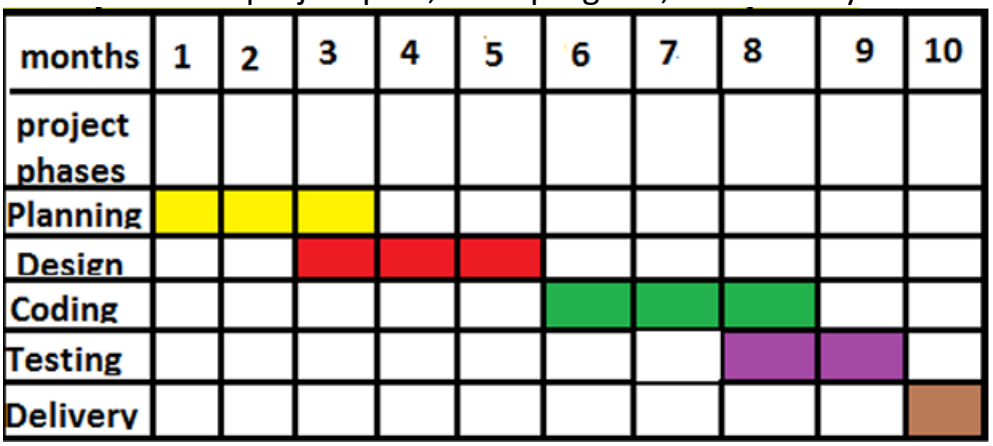
tasks.

• **Scheduling:** They provide a visual timeline for the project, showing when each task is scheduled to start and end, allowing for efficient resource allocation and scheduling.

• **Tracking Progress:** By visually representing the project timeline, Gantt charts allow project managers to easily monitor progress, identify delays, and track completed tasks.

• **Managing Time:** Gantt charts help manage project timelines by highlighting critical paths, dependencies, and potential bottlenecks, enabling timely interventions and adjustments.

• **Communication:** They serve as a communication tool, allowing teams to easily understand the project plan, track progress, and identify areas requiring attention.




• **Requirements Gathering:** Defining project scope, gathering requirements, and documenting specifications.

• **Design:** Designing the software architecture, user interface, and database schema.

• **Development:** Coding the software components, including front-end and back-end development.

• **Testing:** Performing unit testing, integration testing, and user acceptance testing.

• **Deployment:** Deploying the software to the production environment.

• **Maintenance:** Providing ongoing support and maintenance for the software.

11. Describe the steps involved in developing a project schedule in software project management. Include the role of dependencies, durations, resource constraints, and milestones.

A. **1. Define Activities**

- Break down the project into **tasks/phases** (e.g., requirements, design, coding, testing, deployment).
- Each activity should be **clearly defined** with scope and deliverables.

**2. Identify Dependencies**

- Determine **precedence relationships**:
  - *Finish-to-Start (FS)*: Task B starts only after Task A finishes.
  - *Start-to-Start (SS)*: Two tasks can start together.
  - *Finish-to-Finish (FF)*: Two tasks must finish together.
- Dependencies ensure logical sequencing (e.g., testing depends on coding

completion).

**3. Estimate Durations**

- Assign realistic **time estimates** for each activity.
- Techniques: expert judgment, historical data, or models like **COCOMO** for software effort estimation.
- Durations feed into the **critical path analysis**.

**4. Consider Resource Constraints**

- Check availability of **developers, testers, tools, and budget**.
- Resource leveling: adjust start/end dates if resources are over-allocated.
- Sometimes resource limits extend durations even if dependencies allow earlier starts.

**5. Set Milestones**

- Milestones are **zero-duration checkpoints** marking significant events (e.g., "Design Complete," "Beta Release").
- They help track progress and communicate status to stakeholders.
- Milestones are often tied to **deliverables** or contractual obligations.

**6. Construct the Schedule**

- Use tools like **Gantt charts** or **network diagrams (PERT/CPM)**.
- Perform **forward pass** (earliest times) and **backward pass** (latest times).
- Identify **critical path** (longest path, zero float).
- Adjust for resource constraints and milestones.

**7. Review and Baseline**

- Validate schedule with stakeholders.
- Set a **baseline schedule** for tracking progress.
- Future updates compare **actual vs. planned** to manage deviations.

12. Differentiate between WBS, Product Breakdown Structure (PBS), and Resource Breakdown Structure (RBS). Illustrate with diagrams how each breakdown structure aids in planning and control of a software project.

A.

| Aspect | Work Breakdown Structure (WBS) | Product Breakdown Structure (PBS) | Resource Breakdown Structure (RBS) |
|---|---|---|---|
| **Definition** | Hierarchical decomposition of the *work/activities* needed to complete the project | Hierarchical decomposition of the *deliverables/products* of the project | Hierarchical decomposition of the *resources* (human, hardware, software, budget) required |
| **Focus** | *Tasks and activities* | *Outputs and deliverables* | *Inputs/resources* |
| **Purpose** | Defines scope, supports scheduling, assigns responsibilities | Ensures all deliverables are identified and produced | Helps allocate resources, estimate costs, and balance workloads |
| **Example (Software Project)** | Project → Requirements → Design → Coding → Testing → Deployment | Software System → User Interface → Database → API → Documentation | Resources → Human (PM, Developers, Testers) → Hardware (Servers, PCs) → Tools (IDE, Testing Suite) |
| **Planning Role** | Breaks project into manageable tasks for scheduling | Ensures completeness of deliverables | Ensures resource availability and allocation |
| **Control Role** | Tracks progress of tasks and milestones | Tracks delivery of components | Tracks usage of resources and costs |

**WBS (Work Breakdown Structure)**

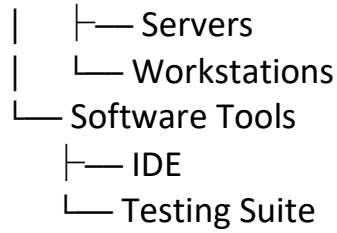
Software Project
- ├— Requirements
- ├— Design
- ├— Coding
- |  ├— Module A
- |  ├— Module B
- ├— Testing
- └— Deployment

**PBS (Product Breakdown Structure)**

Software System
- ├— User Interface
- ├— Database
- ├— API Layer
- └— Documentation

**RBS (Resource Breakdown Structure)**

Resources
- ├— Human
- |  ├— Project Manager
- |  ├— Developers
- |  └— Testers
- ├— Hardware


├── Servers
└── Workstations
└── Software Tools
├── IDE
└── Testing Suite